Learning to Accelerate Receding-Horizon MILP Path Planning

Yunus Emre Kara[1] and Murad Abu-Khalaf[2]

***Abstract*— Accelerating a receding-horizon path planner that solves a mixed-integer linear program (MILP) at each control step is considered, for an environment with multiple vehicles and obstacles. For every step of the planning horizon, a solver must determine the side each vehicle should take relative to the sides of every other vehicle and obstacle to avoid collisions, encoded as binary integer variables. Fixing these binary decisions results in a simpler problem dominated by continuous variables. We therefore propose a predict-and-fix method to predict and partially fix these binary variables to accelerate the MILP solver. The main contribution is a sequence-to-sequence learning method that uses a recurrent neural network, and leverages kinematic heuristics to predict the ride sides for each vehicle-obstacle pair over the planning horizon. We fix a ratio of the vehicle-obstacle binary variables and leave the rest to the solver as a residual MILP problem. If the residual MILP problem is not feasible, the method relaxes some of the earlier fixes. Across held-out instances, with closed-loop replanning, unseen maps, longer horizons, and coupled vehicles, the method consistently reduces the solution time by more than 70%. Moreover, the obtained solutions predominantly attain the optimal or near-optimal solution of the original MILP.**

I. Introduction

Collision-free path planning can be written as a mixed-integer linear program. Schouwenaars et al. showed this for multi-vehicle planning in their 2001 ECC paper [1]. For a rectangular obstacle, the vehicle must remain in at least one of four exterior half-planes. Four binary variables encode these alternatives at every time step. In receding-horizon control, a new MILP is built from the measured state, solved, and only its first control input is applied [2], [3]. The same discrete choices must therefore be solved at every replanning update. This repeated search is expensive even in a modest case, such as an environment with a single vehicle and a dozen obstacles for a planning horizon of five seconds; resulting in an MILP with at least two thousand binary decision variables to optimize for.

Predicting integer decisions before a solve is well-established. Complete or task-specific strategies have been learned for parametric optimization and robot planning [4], [5], [6]. Neural Diving is the closest framework to ours [7]. It predicts values for individual integer variables, learns which predictions to fix, and sends the reduced problems to a solver. Han et al. retain a search neighborhood around a prediction [8]. Yoon studies confidence-based selection and learned coverage in two preprints [9], [10]. More recent methods use contrastive training or alternating prediction and correction to make model reduction more reliable [11], [12]. Other approaches to accelerate MILP solvers include learning-to-branch [13].

Unlike the discussed approaches, our predict-and-fix method leverages the structure of the path-planning formulation directly and introduces kinematic heuristics to facilitate the learning. We introduce a recurrent neural network that we call SideGRU to predict for every time step the best physical side of an obstacle a vehicle is to be at to avoid collisions; namely left, right, below, or above of the obstacle. We rank these predictions and fix the most confident ones, and leave the more uncertain ones to the solver, in this case SCIP [14], [15] to resolve. Since fixing only tightens variable bounds, every feasible plan returned by a reduced problem also satisfies the original MILP. If a wrong prediction makes the residual MILP infeasible, the method relaxes earlier fixes and, if needed, returns to the full MILP.

In Sec. II, we introduce the MILP that is to be solved at each re-planning cycle of the receding-horizon approach. We also motivate the selection of the variables of the MILP that are the learning targets. In Sec. III, we present the proposed predict-and-fix method showing how the dataset is constructed and detailing the learning methodology and architecture. Sec. IV provides numerical experiments demonstrating the performance of the proposed method, and its ability to generalize longer horizons and larger environments. Sec. V discusses the overall results, and Sec. VI provides conclusions and future directions.

II. Receding-horizon MILP path planning

We consider the formulation of Schouwenaars et al. [1] which instead of solving a constrained optimal control problem with a quadratic cost, it replaces the quadratic cost with a weighted l_1-norm cost function. It then encodes the weighted l_1-norm by the introduction of real-valued slack variables for every time sample to constrain the control inputs and states for the inflated dynamics of the direct transcription formulation. Moreover, it encodes collision avoidance by modeling each obstacle as a rectangular obstacle and introducing integer-valued binary slack variables to convert OR-constraints representing the rectangular obstacles to AND-constraints. It furthermore performs similar steps to ensure no collisions between the vehicles themselves. Without loss of generality, we consider stationary obstacles.

For K vehicles, L stationary obstacles, and a horizon length of N time steps, the formulation in [1] leads to the following MILP in eq. (1), where the k-th vehicle state $x^{(k)} \in \mathbb{R}^4$ and control $u^{(k)} \in \mathbb{R}^2$ are given by $x^{(k)} = [p_x^{(k)} \;\; p_y^{(k)} \;\; \dot{p}_x^{(k)} \;\; \dot{p}_y^{(k)}]^\intercal$ and $u^{(k)} = [u_x^{(k)} \;\; u_y^{(k)}]^\intercal$ respectively.

[1]Faculty of Aeronautics and Astronautics, Istanbul Technical University, Istanbul, Türkiye (e-mail: karay22@itu.edu.tr).

[2]Turkish Aerospace, Ankara, Türkiye (abukhalaf@ieee.org).

Moreover, $\sigma^{(k)}[n] \in \mathbb{R}^4$ is a slack variable associated with the l_1-norm of the state tracking error for the state of the k-th vehicle at time step n (1b). Similarly, $\mu^{(k)}[n] \in \mathbb{R}^2$ is a slack variable associated with the l_1-norm of the control input (1c). The slack variable $\alpha_{k,o}[n] \in \mathbb{R}^4$ in eq. (1g) encodes collision avoidance between the k-th vehicle and the o-th obstacle. Similarly, the slack variable $\beta_{k,l}[n] \in \mathbb{R}^4$ in eq. (1h) encodes collision avoidance between the k-th and the l-th vehicle. Equations (1d) to (1f) respectively impose state bounds, input bounds, and enforce the vehicle dynamics.

$$J^* = \min_{\substack{\{x,u\} \\ \{\sigma,\mu,\alpha,\beta\}}} \sum_{k=1}^{K} \Bigg(\sum_{n=1}^{N-1} q^{(k)\intercal} \sigma^{(k)}[n] + \sum_{n=0}^{N-1} r^{(k)\intercal} \mu^{(k)}[n] + q_f^{(k)\intercal} \sigma^{(k)}[N] \Bigg) \tag{1a}$$

subject to: $\forall k \in \{1, \dots, K\}$:

$$\begin{aligned} \forall n \in \{1, \dots, N\} : \; p_x^{(k)}[n] - p_{xf}^{(k)} &\leq \sigma_1^{(k)}[n] \\ -p_x^{(k)}[n] + p_{xf}^{(k)} &\leq \sigma_1^{(k)}[n] \\ p_y^{(k)}[n] - p_{yf}^{(k)} &\leq \sigma_2^{(k)}[n] \\ -p_y^{(k)}[n] + p_{yf}^{(k)} &\leq \sigma_2^{(k)}[n] \\ \dot{p}_x^{(k)}[n] - \dot{p}_{xf}^{(k)} &\leq \sigma_3^{(k)}[n] \\ -\dot{p}_x^{(k)}[n] + \dot{p}_{xf}^{(k)} &\leq \sigma_3^{(k)}[n] \\ \dot{p}_y^{(k)}[n] - \dot{p}_{yf}^{(k)} &\leq \sigma_4^{(k)}[n] \\ -\dot{p}_y^{(k)}[n] + \dot{p}_{yf}^{(k)} &\leq \sigma_4^{(k)}[n] \end{aligned} \tag{1b}$$

$$\begin{aligned} \forall n \in \{0, \dots, N-1\} : \; u_x^{(k)}[n] &\leq \mu_1^{(k)}[n] \\ -u_x^{(k)}[n] &\leq \mu_1^{(k)}[n] \\ u_y^{(k)}[n] &\leq \mu_2^{(k)}[n] \\ -u_y^{(k)}[n] &\leq \mu_2^{(k)}[n] \end{aligned} \tag{1c}$$

$$\begin{aligned} \forall n \in \{1, \dots, N\} : \; x^{(k)}[n] &\geq x_{min}^{(k)} \\ x^{(k)}[n] &\leq x_{max}^{(k)} \end{aligned} \tag{1d}$$

$$\begin{aligned} \forall n \in \{0, \dots, N-1\} : \; u^{(k)}[n] &\geq u_{min}^{(k)} \\ u^{(k)}[n] &\leq u_{max}^{(k)} \end{aligned} \tag{1e}$$

$$\begin{aligned} \forall n \in \{0, \dots, N-1\} : \\ x^{(k)}[n+1] = A^{(k)} x^{(k)}[n] + B^{(k)} u^{(k)}[n] \\ x^{(k)}[0] : \text{init. cond.} \end{aligned} \tag{1f}$$

$$\begin{aligned} \forall n \in \{1, \dots, N\}, \forall o \in \{1, \dots, L\} : \\ p_x^{(k)}[n] \leq x_o^- + M_1 \left(1 - \alpha_{k,o,1}[n]\right) \\ p_x^{(k)}[n] \geq x_o^+ - M_1 \left(1 - \alpha_{k,o,2}[n]\right) \\ p_y^{(k)}[n] \leq y_o^- + M_1 \left(1 - \alpha_{k,o,3}[n]\right) \\ p_y^{(k)}[n] \geq y_o^+ - M_1 \left(1 - \alpha_{k,o,4}[n]\right) \\ \alpha_{k,o,j}[n] \in \{0,1\} : 2 \geq \sum_{j=1}^{4} \alpha_{k,o,j}[n] \geq 1 \end{aligned} \tag{1g}$$

$$\begin{aligned} \forall n \in \{1, \dots, N\}, \forall l \in \{k+1, \dots, K\} : \\ p_x^{(k)}[n] - p_x^{(l)}[n] \geq d_x - M_2 \left(1 - \beta_{k,l,1}[n]\right) \\ p_x^{(l)}[n] - p_x^{(k)}[n] \geq d_x - M_2 \left(1 - \beta_{k,l,2}[n]\right) \\ p_y^{(k)}[n] - p_y^{(l)}[n] \geq d_y - M_2 \left(1 - \beta_{k,l,3}[n]\right) \\ p_y^{(l)}[n] - p_y^{(k)}[n] \geq d_y - M_2 \left(1 - \beta_{k,l,4}[n]\right) \\ \beta_{k,l,j}[n] \in \{0,1\} : 2 \geq \sum_{j=1}^{4} \beta_{k,l,j}[n] \geq 1. \end{aligned} \tag{1h}$$

$r^{(k)} \in \mathbb{R}^2_{>0}$ and $q_f^{(k)} \in \mathbb{R}^4_{>0}$ in (1a) are vectors of positive elements. In this work, we penalize the state tracking error at the terminal time step N only, by setting $q^{(k)} = 0_{4\times1}$. Therefore in (1b), only the constraints of the N-th time step and decision variable $\sigma^{(k)}[N]$ are kept, and are omitted for $n = \{1, \dots, N-1\}$.

x_o^- and x_o^+ in (1g) are the left and right edge positions respectively of the o-th obstacle. Similarly, y_o^- and y_o^+ in (1g) are the bottom and upper edge positions respectively of the o-th obstacle. Furthermore, the initial state of each vehicle $x^{(k)}[0]$ in (1f) is assumed collision-free. d_x and d_y in (1h) are the safe inter-vehicle distances. M_1 and M_2 in eqs. (1g) and (1h) are chosen to be sufficiently large positive numbers given the spatial scale of the problem to allow any of the OR-constraints to be satisfied ($\alpha_{k,o,j}[n] = 1$) or relaxed ($\alpha_{k,o,j}[n] = 0$). Moreover, the feasible case of a vehicle covered by two active faces of an obstacle at its corner is allowed by eq. (1g).

Figure 1 shows a single, and multi-vehicle examples.

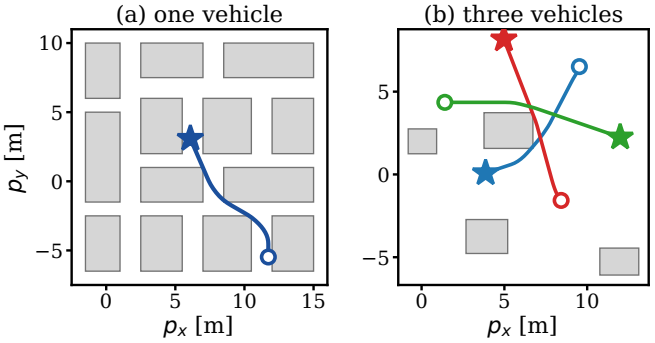


Fig. 1. (a) A single-vehicle instance solved to optimality, having 520 four-way side choices, one per obstacle and step. (b) A three-vehicle crossing used in the transfer study, on a sparser map that the unrestricted baseline can solve within its time limit. Open circles mark starts, stars mark goals.

In the receding-horizon setting, MILP instances are repeatedly solved. The first control input of the planned N time steps horizon is applied, and the state advances as a result. A new MILP instance similar to (1) is solved using the new measurable state, which is then used to initialize the next MILP instance.

A. Selection of Prediction Targets

For each vehicle-obstacle pair (k, o) at time step n, the four binary variables $\{\alpha_{k,o,j}[n]\}_{j=1}^{4}$ form what we call an *obstacle-side group*, where there are $K \times L \times N$ such groups.

Given a vehicle-obstacle pair (k, o), predicting the binary variables $\{\alpha_{k,o,j}[n]\}_{j=1}^{4}$ sequence over time will be the focus of Sec. III and beyond. The hypothesis of this work is that learning to predict this sequence will accelerate the solution of an MILP.

To demonstrate this, assume we have an oracle that can predict this optimal sequence. Moreover, consider the single-vehicle simplified example in Fig. 1, where $K = 1$, $L = 13$, and $N = 40$ corresponding to a 4 s horizon sampled at $\Delta t = 0.1$ s. This results in 520 obstacle-side groups with $4KLN = 2{,}080$ binary variables. There are additionally 4 σ slack variables, 80 μ slack variables, 160 state x variables, and 80 control u variables totaling 324 continuous variables. We used $M_1 = 800$ and $M_2 = 25$.

To characterize the computational cost of the obstacle-side choices, SCIP solves the full MILP in 1.64 s on average over 100 MILP instances all having the same objective but differ in the initial state, the goal state and the obstacle bounds. When the obstacle-side groups are optimally fixed by an oracle, the same solver solves the remaining Linear Program (LP) in 87 ms. This shows that learning to predict the side decisions is a useful target for this problem class.

Remark 2.1: For $K > 1$, without loss of generality, we do not learn to predict the binary variables $\{\beta_{k,l,j}[n]\}_{j=1}^{4}$ in eq. (1h) associated with inter-vehicle spacings. There are $\binom{K}{2}N$ such vehicle-to-vehicle groups that are left to the solver to assign.

III. Predict-and-fix method

Motivated by Sec. II-A, we propose a predict-and-fix (P&F) method, which fixes as many of the obstacle-side groups as possible. For training, we use an optimal solution of an MILP to extract the optimal binary sequences for each vehicle-obstacle pair as training labels. We train a Gated Recurrent Unit (GRU) neural network to learn to generate these sequences given the state of the vehicles, the obstacles and desired goal state as an observation, which we call SideGRU. At deployment time, we fix those obstacle-side groups for whom we have the highest confidence. Secs. III-B to III-D describe these steps in their order of use.

A. Related Work

Our method is related to Neural Diving [7], but differs from it in a major way. We leverage the temporal structure of the problem that is embedded in the MILP instance to perform learning without having to use a bipartite graph representation of the MILP. Note that in a bipartite graph representation as in [13], the input feature vector for each obstacle-side binary decision variable has a zero cost coefficient, since these binary variables do not contribute to the objective function.

Furthermore, our method is related to CoCo which learns discrete planning strategies [5], [16] to convexify an MILP. It is also related to PRISM [6] which combines recurrent prediction with iterative presolve to prune binary variables. In our setup, the semantic of the obstacle-side group binary variables are tied to obstacles, and thus a pruning strategy is not highly plausible.

B. Constructing the Training Dataset

Let $\mathcal{M}$ be a planning MILP instance and $\mathcal{G}$ be the set of all obstacle-side groups. As discussed in Sec. II-A, the optimal values of $\{\alpha_{k,o,j}[n]\}_{j=1}^{4}$ for each obstacle-side group $g = (k, o, n)$ is the learning target; which we extract by solving instance $\mathcal{M}$ to optimality. Therefore, for each instance $\mathcal{M}$, we consider the optimal solutions of the MILP only, unlike what is done in Neural Diving [7] which learns from a pool of feasible solutions weighted by their objective values [7, Sec. 6.1].

As seen in (1g), for each obstacle-side group $g = (k, o, n)$, at most two of the binaries $\{\alpha_{k,o,j}[n]\}_{j=1}^{4}$ can be `true`. If the solver sets two of the binaries of an obstacle-side group $g = (k, o, n)$ to `true`, it is sufficient to consider only one of them as `true` and set the other to `false` without affecting the feasibility or optimality of the solution. Note that `false` simply means the associated OR-constraint is relaxed and ignored, not negated. Therefore, considering a one-hot encoding representation of the target for each obstacle-side group $g = (k, o, n)$ simplifies the learning problem. To decide which side the one-hot encoding represents, we use the signed distances between the optimal position of vehicle k at n and the four boundaries of obstacle o:

$$\begin{aligned} d_g^{\mathrm{L}} &= x_o^- - p_x^{(k)\star}[n], \quad d_g^{\mathrm{R}} = p_x^{(k)\star}[n] - x_o^+, \\ d_g^{\mathrm{D}} &= y_o^- - p_y^{(k)\star}[n], \quad d_g^{\mathrm{U}} = p_y^{(k)\star}[n] - y_o^+, \end{aligned} \tag{2}$$

where the superscripts L, R, D, U correspond to left, right, down, and up respectively. Nonnegative distances correspond to the binaries that are `true`. We therefore encode the side with the largest nonnegative distance

$$y_g = \mathbf{1}\left(\operatorname*{arg\,max}_{c \in \{\mathrm{L,R,D,U}\}} d_g^c\right) \in \mathbb{R}^4, \tag{3}$$

where $\mathbf{1}()$ is the indicator function, $y_g \in \{e_1, e_2, e_3, e_4\}$. $e_i \in \mathbb{R}^4$ is an elementary unit vector with 1 at the i-th index, where $e_1 = \mathbf{1}(L)$, $e_2 = \mathbf{1}(R)$, $e_3 = \mathbf{1}(D)$, and $e_4 = \mathbf{1}(U)$. Moreover, ties are resolved by the listing order.

At this point, our dataset contains a set of MILP instances. For each such instance $\mathcal{M}$, there are KL target sequences where each such target sequence is given by $\{y_{k,o,n}\}_{n=1}^{N}$.

To learn the target sequence $\{y_{k,o,n}\}_{n=1}^{N}$ resulting from (3), we need to find a representation of the input MILP instance $\mathcal{M}$. In this work, we do not use a bipartite graph representation of $\mathcal{M}$ which would require us to provide input features for each graph vertex. Instead, we use the initial data used to construct the MILP itself to generate for each vehicle–obstacle pair (k, o) a sequence of input features $\{f_{k,o,n}\}_{n=1}^{N}$, where $f_{k,o,n} \in \mathbb{R}^{28\times 1}$ is a vector of temporal heuristics interpolating between the initial and goal state as defined in Table I. An instance $\mathcal{M}$ with K vehicles

and L obstacles therefore supplies KL such input feature sequences.

It is important to note that computing $f_{k,o,n}$ does not require solving for the MILP itself. Instead, $f_{k,o,n}$ is computed separately from the initial state of the k-th vehicle $p_0^{(k)} = [p_x^{(k)}[0],\ p_y^{(k)}[0]]^\top$ and $v_0^{(k)} = [\dot{p}_x^{(k)}[0],\ \dot{p}_y^{(k)}[0]]^\top$, the goal position $p_f^{(k)} = [p_{xf}^{(k)}, p_{yf}^{(k)}]^\top$, edge positions ($x_o^-$, x_o^+, y_o^-, y_o^+), center ($c_{x,o}$, $c_{y,o}$), width (w_o) and height (h_o) of the o-th obstacle.

TABLE I

INPUT FEATURE VECTOR $f_{k,o,n} \in \mathbb{R}^{28\times 1}$.

Features	Dim.	Entries and definition
Horizon	2	$\tau_n = \frac{n}{N},\quad t_n = n\Delta t$
Measured initial state	4	$p_x^{(k)}[0],\ p_y^{(k)}[0],\ \dot{p}_x^{(k)}[0],\ \dot{p}_y^{(k)}[0]$
Goal state	2	$p_{xf}^{(k)},\ p_{yf}^{(k)}$
Obstacle geometry	8	$x_o^-,\ x_o^+,\ y_o^-,\ y_o^+,\ c_{x,o},\ c_{y,o},\ w_o,\ h_o$
Linear interpolation	2	$r_{k,n}^{\text{line}} = p_0^{(k)} + \tau_n(p_f^{(k)} - p_0^{(k)})$
Constant-velocity linear interpolation	2	$r_{k,n}^{\text{cv}} = p_0^{(k)} + t_n v_0^{(k)}$
Face offsets	4	$r_{x,k,n}^{\text{line}} - x_o^-,\quad x_o^+ - r_{x,k,n}^{\text{line}},$ $r_{y,k,n}^{\text{line}} - y_o^-,\quad y_o^+ - r_{y,k,n}^{\text{line}}$
Center offsets	2	$r_{x,k,n}^{\text{line}} - c_{x,o},\quad r_{y,k,n}^{\text{line}} - c_{y,o}$
Goal displacement	2	$p_{xf}^{(k)} - p_x^{(k)}[0],\quad p_{yf}^{(k)} - p_y^{(k)}[0]$
Total	28	

At this point, our dataset contains a set of MILP instances. For each such instance $\mathcal{M}$, there are KL pairs of input-target sequences $(\{f_{k,o,n}\}_{n=1}^N,\ \{y_{k,o,n}\}_{n=1}^N)$.

C. Learning Methodology

It is not possible to assume that the mapping between $f_{k,o,n}$ and $y_{k,o,n}$ is static because the targets $y_{k,o,n}$ are coupled to other vehicle-obstacle and vehicle-vehicle pairs, creating a scenario with partial observability of the state of the environment. Therefore, to learn the input-target $(\{f_{k,o,n}\}_{n=1}^N,\ \{y_{k,o,n}\}_{n=1}^N)$ sequences across our dataset, we will treat the problem as a sequence-to-sequence learning problem. We propose the use of a bidirectional gated recurrent unit (GRU) that we refer to as SideGRU. It uses an initial embedding stage using an a two-layer perceptron $28 \to 80 \to 80$ with ReLUs to encode input feature vectors to an embedding space of dimension 80. This then feeds into a 2-layer stacked bidirectional GRUs, each layer has 96 units per direction for a total of 384 individual neurons, and using a 0.1 dropout. A 1-layer perceptron then performs a final embedding of the 192 output of the GRU at every step n, resulting in 4 dimensional logits to which softmax is applied to get a $\mathbb{R}^{N\times 4}$ output.

We use a categorical cross-entropy loss for training and run AdamW solver for 50 epochs. Moreover, a 10^{-3} learning rate, weight decay of 10^{-4}, mini-batch size of 128 sequences, and gradient clipping at 5 are used.

D. Deployment

At deployment time, we consider the design parameter $\rho \in [0, 1]$ which is the fraction of predicted targets that are not fixed and remain free, available to the solver to determine in the residual MILP. If fixing all predicted targets leads to infeasibility, then we free some of them to recover. One may use bi-sectioning search strategies to identify the appropriate value of ρ.

To guide the selection process, we rank the $\mathbb{R}^{N\times 4}$ output of the SideGRU for every obstacle-side group $g = (k, o, n)$. Namely, for each $g = (k, o, n)$, we record the predicted side ($\hat{s}_g$) and its probability (p_g) given by:

$$\hat{s}_g = \underset{c\in\{\text{L,R,D,U}\}}{\arg\max}\ \pi_g^c, \qquad p_g = \max_{c\in\{\text{L,R,D,U}\}} \pi_g^c. \tag{4}$$

Noting that $\mathcal{G}$ is the set of all obstacle-side groups, we let $G = |\mathcal{G}|$ denote the total number of these groups. Given a desired value for ρ, the number of groups to be free is $n_{\text{free}} = \text{round}(\rho G)$ and those to be fixed are $n_{\text{fixed}} = G - n_{\text{free}}$. Among subsets all $\mathcal{F} \subseteq \mathcal{G}$ whose size is $|\mathcal{F}| = n_{\text{fixed}}$, we select the groups that has the largest sum

$$\mathcal{F}_\rho \in \underset{|\mathcal{F}|=n_{\text{fixed}}}{\arg\max} \sum_{g\in\mathcal{F}} p_g. \tag{5}$$

Equation (5) can be simply satisfied by sorting with respect to p_g from highest to lowest, and fixing the first n_{fixed} elements of the sorted list. The weakest link in this approach is if the group $g = (k, o, n)$ with the highest p_g value has a $\hat{s}_g$ that causes infeasibility. In Sec. IV-C, we examine and compare with the case when group selections is done randomly.

Note that our fixing strategy therefore is different than what is done in Neural Diving which jointly learns a selector using SelectiveNet [7], [17]. It is also different from the use of a confidence cutoff [9]; or the use of a threshold-aware learning to learn coverage from solver outcomes [10]; or the ranking of probabilities to select counts of zero and one predictions [8]. Our approach handles the four binary variables of each obstacle-side group $g = (k, o, n)$ together, and hence preserves this structural knowledge of the problem, and does not use additional selector training.

IV. EXPERIMENTS

We test how much the proposed predict-and-fix method reduces planning time and how it affects solution quality. We first use selection tuning problems to choose how many obstacle-side groups to leave free. We then test the method on test problems, examine the role of prediction confidence, and evaluate closed-loop replanning and transfer to other planning settings.

A. Experimental setup

We solve each planning problem with both the original MILP and the proposed method. In the original MILP, SCIP determines all decisions. In P&F, selected obstacle-side groups are fixed to the SideGRU predictions, and SCIP determines the remaining decisions. We compare the time taken and the objective value obtained by each approach.

The main experiments use the single-vehicle map with 13 obstacles shown in Fig. 1. The planning horizon is $4\,\mathrm{s}$, sampled at $0.1\,\mathrm{s}$ intervals, giving 40 planning steps. The obstacles remain fixed, while the vehicle's start and goal positions vary across problems.

The training data include MILP instances in which the vehicle starts at rest, as well as instances collected during closed-loop SCIP simulations. The simulation data capture the positions and velocities encountered as the vehicle moves. This allows the model to learn from both zero and non-zero initial velocities. We obtain training labels from problems solved to optimality by SCIP.

Training runs for a fixed 50 epochs, without early stopping. The selection tuning and test problems are not used for training. We use the selection tuning set to choose the fraction of groups left free and the recovery settings. We then evaluate these choices on the test set. The same trained model and recovery settings are used in all subsequent experiments.

All problems are solved with SCIP 10.0 [15] through PySCIPOpt on an AMD Ryzen 7 6800H processor. Both approaches use the same solver settings, with presolve and cut separation disabled.

The reported times include MILP construction. For P&F, they also include feature computation, network inference, and any additional attempts needed to obtain a plan. If recovery reaches the original MILP, we add the reference solve time measured for the same problem.

We evaluate solution quality against the optimal objective value of the original MILP, denoted by $J^\star$. We count a solution as matching this value when

$$|J_{\mathrm{P\&F}} - J^\star| \le 10^{-4}(|J^\star| + 1). \tag{6}$$

This comparison is made after solving the problems. During deployment, P&F accepts a feasible plan without needing to know $J^\star$.

B. Choosing the free fraction and held-out results

Fixing more groups leaves fewer discrete decisions to the solver. However, incorrect predictions can prevent the solver from finding a feasible plan. Additional solves may then be needed, so fixing more groups does not always reduce total planning time.

We use the selection tuning set to choose ρ, the fraction of obstacle-side groups left free. For each tested value, we first solve the residual MILP. If this attempt returns no feasible plan, we add to it the cost of solving the original MILP.

As shown in Table II, fixing all groups returns a motion plan on only 67 of the 100 problems. Leaving 20% free raises this number to 99 and gives the lowest mean time among the tested values. Larger free fractions allow more choices but take longer to solve. We therefore choose $\rho = 0.2$ for the first attempt to create a residual MILP.

TABLE II

SELECTION TUNING RESULTS ON 100 PROBLEMS. MEAN P&F TIME INCLUDES THE FULL-MILP SOLVE AFTER AN INITIAL FAILURE. THE FULL-MILP MEAN IS 1.64 S. INITIAL PLAN COUNTS PROBLEMS WITH A FEASIBLE PLAN FROM THE FIRST RESIDUAL SOLVE. FINAL MATCH COUNTS FINAL SOLUTIONS MATCHING $J^\star$ ACCORDING TO (6), INCLUDING FULL-MILP FALLBACKS.

ρ	P&F [s]	reduction	initial plan	final match
0.0	1.10	32.8%	67/100	64/100
0.1	0.47	71.0%	93/100	84/100
0.2	**0.42**	**74.6%**	**99/100**	**91/100**
0.3	0.55	66.7%	100/100	93/100
0.4	0.81	50.7%	100/100	98/100

If the first attempt returns no plan, we release more groups by increasing ρ to 0.3. This is the smallest tested value above 0.2 that gives a plan on every selection tuning problem. If this second attempt also returns no plan, we remove all fixes and solve the original MILP. We use this $0.2 \to 0.3 \to 1$ recovery sequence in the remaining experiments.

TABLE III

TEST RESULTS ON 100 PROBLEMS WITH THE SELECTED RECOVERY SETTINGS. PLAN AND MATCH COUNTS REFER TO P&F. MATCHES SATISFY (6).

measure	result
Full-MILP mean time [s]	1.49
P&F mean time [s]	0.31
Time reduction	79.1%
95% confidence interval for reduction	72.2–84.0%
Problems solved faster by P&F	87/100
Feasible plans on the first attempt	97/100
Final feasible plans	100/100
Final matches with $J^\star$	87/100

As shown in Table III, mean planning time falls from $1.49\,\mathrm{s}$ for the original MILP to $0.31\,\mathrm{s}$ for P&F. This is a 79.1% reduction, with a 95% paired bootstrap confidence interval of 72.2–84.0% over problems. The method is faster on 87 of the 100 problems.

The first attempt gives a plan on 97 problems. Releasing more groups gives a plan on two of the remaining three, while one requires the original MILP. All 100 problems finish with a feasible plan, and 87 solutions match $J^\star$. Thus, most problems retain the optimal objective value while requiring less planning time. For the remaining problems, a feasible plan does not necessarily have the optimal objective value.

The method also reduces planning time with SCIP's default settings. In this comparison, the mean falls from 8.45 to $1.47\,\mathrm{s}$, showing that the improvement is also present with presolve and cut separation enabled.

C. The role of prediction confidence

We next test whether selection confidence as discussed in Sec. III-D helps us choose which groups to fix. We use the same predictions and fix the same number of groups in both cases. One selects the groups with the highest confidence;

TABLE IV

CONFIDENCE-BASED AND RANDOM SELECTION ON THE HELD-OUT SET. PLAN AND MATCH RATES REFER TO THE FIRST ATTEMPT. MEAN TIME INCLUDES RECOVERY.

selection	trials	initial plan	initial match	P&F [s]
confidence	100	97.0%	84.0%	0.31
random	500	68.6%	24.4%	0.84

the other selects groups at random. We repeat the random selection five times for each held-out problem.

As shown in Table IV, confidence-based selection gives more feasible plans on the first attempt and preserves the optimal objective value more often. It also reduces mean time, including recovery, from 0.84 to 0.31 s.

Both cases use the same predictions and fix the same number of groups. The difference therefore shows that the choice of groups matters. Prediction confidence helps identify which decisions to fix and which to leave to the solver, although it does not guarantee a feasible residual problem.

D. Closed-loop replanning

We now test the method while it controls a moving vehicle. At each step, the vehicle's new position and velocity define the next planning problem. This allows us to examine whether the time savings continue over successive replanning steps. We run 20 simulations with 30 replanning steps each.

At every step, we solve the same problem with both the original MILP and P&F. We apply only the first control input from the P&F solution. The full MILP therefore provides a reference at the same states visited by the P&F controller. We do not compare trajectories produced by two separate controllers.

TABLE V

CLOSED-LOOP RESULTS OVER 20 SIMULATIONS AND 600 REPLANNING STEPS. PLAN AND MATCH COUNTS REFER TO P&F. MATCHES SATISFY (6).

measure	result
Full-MILP mean time [s]	1.01
P&F mean time [s]	0.22
Time reduction	78.2%
Full-MILP 95th-percentile time [s]	3.27
P&F 95th-percentile time [s]	0.45
Final feasible plans	600/600
Final matches with $J^\star$	540/600

As shown in Table V, mean planning time falls from 1.01 to 0.22 s across the 600 replanning steps, a reduction of 78.2%. The 95th percentile falls from 3.27 to 0.45 s, so the method also reduces the longer planning times.

When the first attempt returns no plan, we release more groups and, if needed, return to the original MILP. All 600 steps give a feasible plan. For the same vehicle states, 540 of the 600 solutions match the optimal objective value of the original MILP.

The time savings continue during closed-loop replanning. The measured times, however, do not establish real-time operation at the 0.1 s sampling interval. A more efficient implementation, for example in C++, and faster hardware could reduce planning time and make real-time operation possible.

E. Transfer to other planning settings

Finally, we test the same model on maps and problem sizes that differ from those used for training. We keep the model and recovery settings fixed throughout these tests.

The unseen-map tests use new obstacle layouts with the same horizon and obstacle count as the main experiments. The other tests use a 6 s horizon, denser maps with 17–20 obstacles, or two and three vehicles in maps with four obstacles. In the multi-vehicle tests, SideGRU predicts only vehicle–obstacle groups. SCIP determines all inter-vehicle separation decisions. Some tests change both the map and the problem size, so they do not measure the effect of each change separately.

TABLE VI

TRANSFER RESULTS WITH THE SAME MODEL AND RECOVERY SETTINGS. MEAN TIMES INCLUDE RECOVERY. THE FINAL COLUMN COUNTS SOLUTIONS MATCHING $J^\star$.

setting	n	SCIP [s]	P&F [s]	initial plan	final match
unseen maps	20	0.65	0.19	20/20	11/20
$T = 6$ s	20	4.99	0.53	19/20	14/20
dense maps	20	2.69	0.63	19/20	13/20
2 vehicles	30	0.97	0.30	30/30	25/30
3 vehicles	20	15.14	5.96	20/20	17/19

As shown in Table VI, P&F reduces mean planning time in every setting. All 110 problems give a feasible plan after recovery when needed. One three-vehicle reference solve reaches the 120 s limit without proving optimality. We exclude that problem from the optimum-match count but retain its capped time in the timing mean.

The time savings continue across these settings, but the rate of matching the optimum changes. For example, only 11 of the 20 unseen-map solutions match $J^\star$. The same model can therefore reduce planning time beyond the training setting, but solution quality is not preserved to the same extent in every setting.

V. DISCUSSION

The results show that obstacle-side decisions are a useful learning target for reducing planning time. The oracle experiment in the earlier sections showed that knowing these decisions can substantially shorten the solve time. The proposed method learns to provide some of these decisions before the solver starts. However, leaving uncertain decisions to the solver gives better results than fixing all predictions.

The choice of groups matters as much as the number of groups fixed. The confidence-based and random selections use the same predictions and fix the same number of groups. Yet confidence-based selection gives more feasible plans, preserves the optimal objective value more often, and reduces total planning time. Prediction confidence is therefore useful for deciding which groups to fix and which to leave to the solver.

An incorrect fix can prevent the solver from finding a feasible plan or lead to a plan with a higher objective

value. When no plan is found, releasing groups gives the solver more choices. If needed, all fixes are removed and the original MILP is solved. However, the method accepts a plan once it is feasible, so recovery does not guarantee optimality. Returning to the original MILP also requires that the problem be feasible and that the solver finds a plan within the available time.

The same SideGRU model reduces planning time in different planning settings, showing that its use can extend beyond the training environment. However, the lower rate of matching the optimum in some settings shows that transfer is more limited in terms of solution quality. Training on more varied maps and planning states could help reduce this difference. Further experiments are needed to determine how much this would improve the results.

The current results come from a Python and PySCIPOpt implementation. A more efficient implementation, for example in C++, and faster hardware could reduce planning time and make real-time operation possible. The effects of computation delay and modeling errors should also be evaluated for use on a physical system.

VI. Conclusion

We presented a predict-and-fix method that learns obstacle-side decisions to accelerate receding-horizon MILP path planning. SideGRU predicts these decisions over the planning horizon. The method fixes the predictions with the highest confidence and leaves the remaining decisions to SCIP. When no feasible plan is found, it releases additional groups.

Mean planning time decreases by 79.1% on the test problems and by 78.2% during closed-loop replanning. All tested problems return a feasible plan, and most solutions retain the optimal objective value. Confidence-based selection helps choose which decisions to fix, and the same model also reduces planning time in other planning settings. Future work can examine more varied training data to improve solution quality and develop the method further for real-time applications.

Acknowledgment

This work grew out of the first author's graduation project at Istanbul Technical University.

References

[1] T. Schouwenaars, B. De Moor, E. Feron, and J. How, "Mixed integer programming for multi-vehicle path planning," in *2001 European Control Conference (ECC)*, 2001, pp. 2603–2608.

[2] J. Bellingham, A. Richards, and J. How, "Receding horizon control of autonomous aerial vehicles," in *Proceedings of the 2002 American Control Conference*, vol. 5, 2002, pp. 3741–3746.

[3] Y. Kuwata and J. How, "Receding horizon implementation of milp for vehicle guidance," in *Proceedings of the 2005 American Control Conference*, vol. 4, 2005, pp. 2684–2685.

[4] D. Bertsimas and B. Stellato, "Online mixed-integer optimization in milliseconds," *INFORMS Journal on Computing*, vol. 34, no. 4, pp. 2229–2248, Jul. 2022.

[5] A. Cauligi, P. Culbertson, E. Schmerling, M. Schwager, B. Stellato, and M. Pavone, "Coco: Online mixed-integer control via supervised learning," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1447–1454, 2022.

[6] A. Cauligi, A. Chakrabarty, S. Di Cairano, and R. Quirynen, "PRISM: Recurrent neural networks and presolve methods for fast mixed-integer optimal control," in *Proceedings of The 4th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 168, 2022, pp. 34–46.

[7] V. Nair et al., *Solving mixed integer programs using neural networks*, 2021. arXiv: 2012.13349 [math.OC].

[8] Q. Han et al., "A GNN-guided predict-and-search framework for mixed-integer linear programming," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[9] T. Yoon, *Confidence threshold neural diving*, 2022. arXiv: 2202.07506 [cs.LG].

[10] T. Yoon, J. Choi, H. Yun, and S. Lim, *Threshold-aware learning to generate feasible solutions for mixed integer programs*, 2023. arXiv: 2308.00327 [cs.LG].

[11] T. Huang, A. M. Ferber, A. Zharmagambetov, Y. Tian, and B. Dilkina, "Contrastive predict-and-search for mixed integer linear programs," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, Jul. 2024, pp. 19 757–19 771.

[12] H. Liu et al., "Apollo-MILP: An alternating prediction-correction neural solving framework for mixed-integer linear programming," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[13] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi, "Exact combinatorial optimization with graph convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[14] T. Achterberg, "SCIP: Solving constraint integer programs," *Mathematical Programming Computation*, vol. 1, no. 1, pp. 1–41, Jul. 2009.

[15] C. Hojny et al., "The SCIP optimization suite 10.0," Optimization Online, Tech. Rep., 2025.

[16] A. Cauligi, P. Culbertson, B. Stellato, D. Bertsimas, M. Schwager, and M. Pavone, "Learning mixed-integer convex optimization strategies for robot planning and control," in *2020 59th IEEE Conference on Decision and Control (CDC)*, 2020, pp. 1698–1705.

[17] Y. Geifman and R. El-Yaniv, "SelectiveNet: A deep neural network with an integrated reject option," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 2151–2159.